title: InvenTree 仕様書
ref: c4c493d3fd4364fc99a1c6ee067100cca35b6d53 (master)
generated: 2026-09-24
evidence: ソースコードのみ

InvenTree 仕様書

1. 概要・対象範囲・前提

InvenTree は部品・在庫・製造・購買・販売を一元管理する在庫管理システムである。実装は 2 層に分かれ、業務ロジック・DB スキーマ・バックグラウンド処理・プラグイン実行・帳票生成はすべてバックエンドが保持し、フロントエンドは REST API 経由でのみ通信する (`CONTRIBUTING.md`)。

項目	内容	根拠
バックエンド	Django + Django REST Framework (Python)、非同期処理は Django-Q2	`src/backend/InvenTree/`、`InvenTree/tasks.py`
フロントエンド	React 19 + Mantine 9 + Vite の SPA (Platform UI, 通称 PUI)	`src/frontend/src/`
API バージョン	`INVENTREE_API_VERSION = 550`	`InvenTree/api_version.py:4`
DB	PostgreSQL / MySQL / SQLite	`InvenTree/settings.py`、`config_template.yaml`
認証	セッション (django-allauth headless, browser client)、API トークン、MFA	`InvenTree/urls.py:101-118`、`users/api.py`

対象範囲: `src/backend/InvenTree/` の全 Django アプリと `src/frontend/src/` の画面・ルーティング。**対象外 (言及のみ)**: ドキュメントサイト (`docs/docs/`)、CI 定義 (`.github/`)、コンテナ・インストーラ配布物 (`contrib/`)、翻訳リソース (`locale/`, `src/frontend/src/locales/`)。

前提・検証水準: 本書は master の当該コミットのソースを唯一の根拠として作成した。実行環境への接続手段が本作業環境に無いため実機確認は行っていない (front matter `evidence: ソースコードのみ`)。コメントと実装が食い違う場合は実装を正とする。

2. 機能一覧

機能 ID	名称	入口 (画面 / API)	主要ソース
F-01	部品カタログ管理 (作成・改訂・バリアント)	`/part/:id`, `/api/part/`	`part/models.py:465` Part, `part/api.py`
F-02	部品カテゴリ階層管理	`/part/category/:id`, `/api/part/category/`	`part/models.py:71` PartCategory
F-03	BOM 管理・検証・コピー	`/part/:id` BOM パネル, `/api/bom/`, `api-part-bom-validate`	`part/models.py:3724` BomItem, `part/api.py`
F-04	部品パラメータ / テストテンプレート	`/part/:id`, `/api/part/test-template/`	`common/models.py:2620` ParameterTemplate, `part/models.py:3559`
F-05	部品価格計算 (Pricing)	`/api/part/:id/pricing/`	`part/models.py:2606` PartPricing
F-06	在庫アイテム管理 (登録・編集・変換・分解)	`/stock/item/:id`, `/api/stock/`	`stock/models.py:426` StockItem, `stock/api.py`

機能ID	名称	入口 (画面 / API)	主要ソース
F-07	保管場所階層管理	`/stock/location/:id`, `/api/stock/location/`	`stock/models.py:126` StockLocation
F-08	在庫調整 (追加・削除・棚卸・移動・状態変更・統合)	`api-stock-add/remove/count/transfer/change-status/merge`	`stock/api.py`, `stock/models.py:3222-3400`
F-09	シリアル化・バッチコード採番	`api-stock-item-serialize`, `/api/generate/serial-number/`, `/api/generate/batch-code/`	`stock/models.py:2402` serializeStock, `stock/api.py`
F-10	在庫履歴・テスト結果記録	`/api/stock/track/`, `/api/stock/test/`	`stock/models.py:3716`, `stock/models.py:3801`
F-11	製造オーダー (Build Order) 管理	`/manufacturing/build-order/:id`, `/api/build/`	`build/models.py:86` Build
F-12	製造オーダーへの在庫割当・自動割当・解除	`api-build-allocate/auto-allocate/unallocate/consume`	`build/models.py:1524` allocate_stock, `build/api.py`
F-13	製造出力の生成・完了・スクラップ	`api-build-output-create/complete/scrap/delete`	`build/models.py:1109`, `:1448`
F-14	製造オーダーの状態遷移 (発行・保留・完了・取消)	`api-build-issue/hold/finish/cancel`	`build/models.py:977`, `:1063`
F-15	購買オーダー管理・入荷処理	`/purchasing/purchase-order/:id`, POST `/api/order/po/:id/receive/`	`order/models.py:639` PurchaseOrder, `order/api.py:470-565`
F-16	販売オーダー管理・割当・出荷	`/sales/sales-order/:id`, `api-so-allocate/shipment-ship`	`order/models.py:1386`, `:2623`
F-17	返品オーダー管理 (受領・処置)	`/sales/return-order/:id`, `api-return-order-receive`	`order/models.py:3235` ReturnOrder
F-18	在庫移動オーダー管理	`/stock/transfer-order/:id`, `api-transfer-order-*`	`order/models.py:3674` TransferOrder
F-19	取引先管理 (仕入先・顧客・製造元・住所・連絡先)	`/company/:id`, `/api/company/`	`company/models.py:78` Company
F-20	仕入先部品・価格ブレーク管理	`/purchasing/supplier-part/:id`	`company/models.py:604` SupplierPart
F-21	ユーザー・グループ・ロール権限管理	`/core/user/:id`, `/api/user/`	`users/ruleset.py`, `users/api.py`
F-22	認証 (ログイン・MFA・パスワード再設定・SSO)	`/login`, `/mfa`, `/api/auth/`	`src/frontend/src/router.tsx:229-239`
F-23	システム / ユーザー設定管理	`/settings/system`, `/settings/user`, `/api/settings/`	`common/models.py:1247`, `:1342`
F-24	通知・メール・Webhook	`/notifications`, `/api/notifications/`	`common/models.py:1439-1774`
F-25	帳票・ラベル印刷	`/api/report/`, `/api/label/`	`report/models.py`, `report/api.py`
F-26	データインポート / エクスポート	`/api/importer/`, DataExportMixin	`importer/models.py`, `plugin/base/integration/DataExport.py`
F-27	バーコード走査・リンク・アクション	`/scan`, `/api/barcode/*`	`plugin/base/barcodes/api.py`

機能 ID	名称	入口 (画面 / API)	主要ソース
F-28	プラグイン管理 (インストール・有効化・設定)	`/settings/admin`, `/api/plugins/`	`plugin/api.py`, `plugin/registry.py`
F-29	機械 (プリンタ等) 管理	`/api/machine/`	`machine/api.py`, `machine/models.py`
F-30	全体検索	`/api/search/`	`InvenTree/urls.py:67` APISearchView
F-31	バックグラウンド定期タスク (バックアップ・為替更新・マイグレーション検査等)	Django-Q2 スケジューラ	`InvenTree/tasks.py:655-1160`

3. 画面 (API) フロー

3.1 画面ルーティング (主要のみ)

URL	画面	機能 ID
`/home/`	ダッシュボード	F-30
`/part/category/:id`, `/part/:id`	部品カテゴリ / 部品詳細	F-01〜F-05
`/stock/location/:id`, `/stock/item/:id`	保管場所 / 在庫アイテム詳細	F-06〜F-10
`/stock/transfer-order/:id`	在庫移動オーダー詳細	F-18
`/manufacturing/index`, `/manufacturing/build-order/:id`	製造一覧 / 製造オーダー詳細	F-11〜F-14
`/purchasing/purchase-order/:id`, `/purchasing/supplier-part/:id`	購買オーダー / 仕入先部品	F-15, F-20
`/sales/sales-order/:id`, `/sales/shipment/:id`, `/sales/return-order/:id`	販売オーダー / 出荷 / 返品	F-16, F-17
`/core/user/:id`, `/core/group/:id`	ユーザー / グループ	F-21
`/settings/system`, `/settings/user`, `/settings/admin`	設定 / 管理センター	F-23, F-28
`/scan/`	バーコード走査	F-27
`/login`, `/mfa`, `/reset-password`, `/verify-email/:key`	認証系	F-22

根拠: `src/frontend/src/router.tsx:168-239`。各詳細画面は `/*` のサブパスでパネル (Details, Stock, BOM 等) を切り替える。

3.2 API 名前空間

`/api/` 配下に集約される (`InvenTree/urls.py:54-120`)。

`admin/`, `bom/`, `build/`, `company/`, `importer/`, `label/`, `machine/`, `order/`, `part/`, `report/`, `search/`, `settings/`, `stock/`, `user/`, `generate/{batch-code,serial-number}/`, `schema/`, `version/`, `license/`, `auth/` (allauth headless), プラグイン系 (`plugin/api.py`)。

3.3 代表フロー (テキスト図)

- 購買: 購買オーダー作成 → `POST /api/order/po/:id/issue/` (PENDING→PLACED) → `.../receive/` (入荷: StockItem 生成) → `.../complete/` (COMPLETE)。購買オーダーは DRF ViewSet の `@action` で遷移エンドポイントを提供する (`order/api.py:470-565`, ルータ登録は `order/api.py:576`)。
- 製造: 製造オーダー作成 → `api-build-issue` (PENDING→PRODUCTION) → `api-build-allocate` / `api-build-auto-allocate` → `api-build-output-create` → `api-build-output-complete` → `api-build-finish` (COMPLETE)。
- 販売: 販売オーダー作成 → `api-so-issue` → `api-so-allocate` / `api-so-auto-allocate` → 出荷作成 → `api-so-shipment-ship` → `api-so-complete`。

- 返品: 返品オーダー作成 → `api-return-order-issue` → `api-return-order-receive` → 明細ごとに処置 (ReturnOrderLineStatus) → `api-return-order-complete`。

4. データモデル

モデル (テーブル)	主要列	キー・制約	用途
`Part` (`part/models.py:465`)	name, IPN, revision, revision_of, variant_of, category, assembly, component, trackable, purchaseable, salable, virtual, active, locked, bom_validated	(name, IPN, revision) の一意制約、MPTT ツリー (variant)	部品マスタ
`PartCategory` (`:71`)	name, parent, structural, default_location	MPTT 階層	部品分類
`BomItem` (`:3724`)	part, sub_part, quantity, optional, consumable, allow_variants, substitutes, validated	part-sub_part 一意	BOM 明細
`PartPricing` (`:2606`)	bom_cost_min/max, purchase_cost, internal_cost, sale_price, overall_min/max	Part と 1:1	価格集計
`StockItem` (`stock/models.py:426`)	part, supplier_part, location, quantity, serial, batch, status, expiry_date, belongs_to, build, is_building, consumed_by	MPTT (分割履歴)、シリアルは Part ツリー内一意	在庫実体
`StockLocation` (`:126`)	name, parent, structural, external, location_type	MPTT 階層	保管場所
`StockItemTracking` (`:3716`)	item, date, tracking_type (StockHistoryCode), deltas, user	追記のみ	在庫履歴
`StockItemTestResult` (`:3801`)	stock_item, template, result, value, attachment	-	検査結果
`Build` (`build/models.py:86`)	reference, part, quantity, status, completed, target_date, start_date, external, responsible, sales_order	reference 一意	製造オーダー
`BuildLine` (`:2170`)	build, bom_item, quantity	build-bom_item 一意	所要明細
`BuildItem` (`:2290`)	build_line, stock_item, quantity, install_into	-	在庫割当
`Order` 派生 (`order/models.py:317`)	reference, status, description, responsible, target_date, project_code	reference 一意	PO/SO/RO/TO 共通基底
`PurchaseOrderLineItem` (`:2246`)	order, part (SupplierPart), quantity, received, purchase_price, destination	-	購買明細
`SalesOrderLineItem` / `SalesOrderAllocation` / `SalesOrderShipment` (`:2468`, `:3090`, `:2623`)	line, item, quantity, shipment, shipment_date	出荷単位で割当	販売・出荷
`ReturnOrderLineItem` (`:3560`)	order, item, outcome (ReturnOrderLineStatus), received_date	-	返品明細
`TransferOrderLineItem` / `TransferOrderAllocation` (`:4006`, `:4094`)	order, part, quantity, item	-	在庫移動
`Company` / `SupplierPart` / `ManufacturerPart` (`company/models.py:78`, `:604`, `:489`)	name, is_customer, is_supplier, is_manufacturer / SKU, MPN, pack_quantity	SKU は (supplier, SKU) 一意	取引先・調達
`InvenTreeSetting` / `InvenTreeUserSetting` (`common/models.py:1247`, `:1342`)	key, value (user)	key 一意	設定

モデル (テーブル)	主要列	キー・制約	用途
`Attachment`, `Note`, `Parameter`, `BarcodeScanResult`, `NotificationMessage`, `EmailMessage`, `DataOutput` (`common/models.py`)	各汎用列	-	横断機能

状態コードは `generic/states/StatusCode` 派生で定義され、API から `api-*-status-codes` で取得できる。

状態集合	値	根拠
BuildStatus	10 Pending / 20 Production / 25 On Hold / 30 Cancelled / 40 Complete	`build/status_codes.py:19-23`
PurchaseOrderStatus	10 Pending / 20 Placed / 25 On Hold / 30 Complete / 40 Cancelled / 50 Lost / 60 Returned	`order/status_codes.py:21-27`
SalesOrderStatus	10 Pending / 15 In Progress / 20 Shipped / 25 On Hold / 30 Complete / 40 Cancelled / 50 Lost / 60 Returned	`order/status_codes.py:64-71`
ReturnOrderStatus	10 Pending / 20 In Progress / 25 On Hold / 30 Complete / 40 Cancelled	`order/status_codes.py:100-104`
ReturnOrderLineStatus	10 Pending / 20 Return / 30 Repair / 40 Replace / 50 Refund / 60 Reject	`order/status_codes.py:131-136`
StockStatus	10 OK / 50 Attention needed / 55 Damaged / 60 Destroyed / 65 Rejected / 70 Lost / 75 Quarantined / 85 Returned	`stock/status_codes.py:22-29`

在庫として「利用可能」と扱うのは OK / Attention needed / Damaged / Returned のみ (`stock/status_codes.py:35-40`)。

5. 業務ルール

1. **部品の一意性**: IPN と revision がともに非 NULL の場合、(name, IPN, revision) の重複を拒否する。IPN 重複は設定 `PART_ALLOW_DUPLICATE_IPN` が False のとき拒否する (`part/models.py:1017-1074`)。
2. **改訂 (revision)**: 自身の改訂は不可、改訂には revision コードが必須、テンプレート部品の改訂は不可、`revision_of` は同一 `variant_of` を指す必要がある。設定 `PART_REVISION_ASSEMBLY_ONLY` が有効なら assembly 部品のみ改訂可 (`part/models.py:777-812`)。
3. **IPN 書式**: 設定のパターンが指定されている場合、IPN は正規表現に一致しなければならない (`part/models.py:770-775`)。
4. **構造カテゴリ / 構造ロケーション**: `structural=True` のカテゴリに部品は割当不可、同ロケーションに在庫は配置不可 (`part/models.py:1085-1088`, `stock/models.py:1021-1026`)。
5. **部品削除制限**: locked、active、または BOM で使用中 (`PART_ALLOW_DELETE_FROM_ASSEMBLY` が False のとき) の部品は削除不可 (`part/models.py:593-602`)。
6. **BOM 再帰禁止**: 自己参照・同一バリアントツリー・子 BOM 経由の循環参照を検出して拒否する (`part/models.py:676-720`)。locked された assembly の BOM は変更不可 (`part/models.py:3973`)。
7. **シリアル在庫**: シリアルを持つ在庫は quantity=1 固定 (0 は 1 に補正)、`delete_on_deplete` は強制的に False、シリアル在庫は削除不可 (`stock/models.py:1090-1116`, `:582`)。
8. **シリアル一意性スコープ**: 既定は Part のバリアントツリー内、設定 `SERIAL_NUMBER_GLOBALLY_UNIQUE` が True なら全体一意。並行採番は `select_for_update` でロックし、ロック下で再検証する (`stock/models.py:676-733`)。
9. **シリアル編集**: `STOCK_ALLOW_EDIT_SERIAL` が False の場合、既にシリアルが付与された在庫のシリアル変更は拒否 (`stock/models.py:1038-1055`)。
10. **在庫数量**: 負数は拒否。trackable 部品は整数数量のみ。virtual 部品には在庫を作成できない (`stock/models.py:1061-1079`)。
11. **在庫の整合性**: `supplier_part` は同一 Part を指す必要があり、自己所属は不可、`is_building=True` は build 参照必須、build 出力の Part は Build の Part (または変換候補) と一致する必要がある (`stock/models.py:1081-1139`)。

12. **製造オーダーの完了条件**: status が PRODUCTION、未完了出力なし、残数 0、untracked 明細が全量割当済み。設定 `BUILDORDER_REQUIRE_CLOSED_CHILDS` が有効なら未完了の子オーダーがあってはならない (`build/models.py:640-662`)。
13. **製造オーダーの検証**: 外部委託は purchaseable 部品のみ、`BUILDORDER_REQUIRE_RESPONSIBLE` 有効時は責任者必須、作成後の対象部品変更は不可、target_date は start_date より後 (`build/models.py:216-241`)。
14. **参照番号**: Build / Order の reference はテンプレートに基づき自動採番され一意である (`build/models.py:265-273` reference, `order/models.py:317` Order)。
15. **権限**: 11 のルールセット (admin, part_category, part, bom, stock_location, stock, build, purchase_order, sales_order, return_order, transfer_order) × 4 権限 (view/add/change/delete) をグループに付与する (`users/ruleset.py:26-60`)。
16. **プラグインによる拡張検証**: シリアル番号・バッチコード等は Validation ミックスインを実装したプラグインが先に検証し、例外・None・True で処理を分岐する (`part/models.py:814-860`)。
17. **定期処理**: heartbeat (1 分)、タスク・通知・メール・エラーログの削除、更新確認、為替レート更新、バックアップ、マイグレーション検査を日次で実行する (`InvenTree/tasks.py:655-1160`)。

6. 機能仕様詳細

F-01 部品カタログ管理

- 入口: `/part/:id` 画面、`/api/part/` (list/create)、`/api/part/:id/` (detail)。
- 入力: name (必須), IPN, revision, revision_of, category, description, units, 各種フラグ (assembly / component / trackable / purchaseable / salable / virtual / active / locked)。
- 検証: 業務ルール 1〜5。カテゴリが structural なら拒否。
- 出力: Part レコード。画像差し替え時に旧画像が他部品から参照されていなければ削除する (`part/models.py:606-620`)。
- エラー: 「Duplicate IPN not allowed in part settings」「Part with this Name, IPN and Revision already exists.」等 (7 章)。

F-03 BOM 管理

- 入口: 部品詳細の BOM パネル、`/api/bom/`、`api-part-bom-copy`、`api-part-bom-validate`、`api-bom-item-validate`。
- 入力: part, sub_part, quantity, optional, consumable, allow_variants, inherited, substitutes。
- 検証: sub_part 必須、再帰禁止 (業務ルール 6)、assembly が locked なら変更不可。
- 出力: BomItem。検証操作は `validated` フラグと Part の `bom_validated` を更新する。

F-08 在庫調整

- 入口: `api-stock-add` / `api-stock-remove` / `api-stock-count` / `api-stock-transfer` / `api-stock-change-status` / `api-stock-merge` / `api-stock-assign`。
- 入力: 対象アイテム配列 (pk, quantity)、移動先 location、notes、status。
- 処理: `add_stock` / `take_stock` / `stocktake` / `move` が数量を更新し、`StockItemTracking` に `StockHistoryCode` 付きで履歴を追記する (`stock/models.py:3055-3400`)。
- 検証: 数量 > 0、移動先が structural でないこと、在庫が存在すること。
- エラー: 「Quantity must be greater than zero」「Stock item no longer exists」(`stock/models.py:1494-1504`)。

F-09 シリアル化・採番

- 入口: `api-stock-item-serialize`、`/api/generate/serial-number/`、`/api/generate/batch-code/`、`api-part-serial-number-detail`。
- 入力: quantity, serial_numbers (範囲記法可), destination。

- 処理: `serializeStock` が数量を分割し、シリアルごとに StockItem を生成する。`_create_serial_numbers` はトランザクション内でロックし重複を再検証する。
- エラー: 「The following serial numbers already exist or are invalid : <一覧>」(`stock/models.py:730-733`)。

F-11〜F-14 製造オーダー

- 入口: `/manufacturing/build-order/:id`、`/api/build/`、`api-build-issue/hold/finish/cancel`、`api-build-allocate/auto-allocate/unallocate/consume`、`api-build-output-create/complete/scrap/delete`。
- 入力順: オーダー作成 (part, quantity, target_date, responsible, external) → 発行 → 明細への在庫割当 (build_line, stock_item, quantity) → 出力生成 (quantity, serial_numbers, batch) → 出力完了 (location, status) → 完了。
- 検証: 業務ルール 12・13。割当は `allocate_stock` が明細単位で検証する (`build/models.py:1524`)。
- 出力: 完了時に割当在庫を消費し、出力 StockItem の `is_building` を解除する (`build/models.py:977-1060`)。`trim_allocated_stock` 指定時は余剰割当を削除する。
- 取消: `cancel_build` が割当を解除し status を CANCELLED にする (`build/models.py:1063`)。

F-15 購買オーダー・入荷

- 入口: `/purchasing/purchase-order/:id`、`POST /api/order/po/:id/{hold,cancel,complete,issue,receive}/` (`order/api.py:480-560`)。
- 入力: 明細 (SupplierPart, quantity, purchase_price, destination)、入荷時は line_items 配列 + `batch_code` (トップレベル指定可, API v550 で追加) + status + location。
- 出力: 入荷数量 (`received`) の加算と StockItem 生成、在庫履歴の記録。
- 状態: PENDING → PLACED → COMPLETE (ON_HOLD / CANCELLED / LOST / RETURNED へ分岐)。

F-16 販売オーダー・出荷

- 入口: `/sales/sales-order/:id`、`api-so-issue/hold/cancel/complete`、`api-so-allocate`、`api-so-allocate-serials`、`api-so-auto-allocate`、`api-so-shipment-ship`。
- 入力: 明細 (part, quantity, sale_price)、割当 (line, stock_item, quantity, shipment)、出荷 (shipment_date, tracking_number)。
- 検証: 割当は出荷単位に紐づき、割当数量は在庫数量を超えられない (`order/models.py:3090` SalesOrderAllocation)。
- 出力: 出荷実行で在庫を顧客へ移管し、全出荷完了で COMPLETE に遷移可能。

F-21/F-22 ユーザー・認証

- 入口: `/login`, `/mfa`, `/reset-password`, `/set-password`, `/verify-email/:key`、`/api/auth/` (allauth headless browser client)、`/api/user/`。
- 入力: ユーザー名 / パスワード、MFA コード、SSO プロバイダ選択。
- 権限: グループに付与したルールセットから Django パーミッションを生成する (`users/ruleset.py`)。
- 出力: セッション Cookie。API トークンは `/api/user/token/` 系で発行する。

F-27 バーコード

- 入口: `/scan/` 画面、`api-barcode-scan`、`api-barcode-link`、`api-barcode-unlink`、`api-barcode-generate`、`api-barcode-po-receive`、`api-barcode-po-allocate`、`api-barcode-so-allocate`。
- 処理: 登録済みバーコードプラグインを順に評価し、一致したモデルインスタンスを返す。走査結果は `BarcodeScanResult` に保存する (`common/models.py:3447`)。

F-28 プラグイン

- 入口: `/settings/admin`、`api-plugin-list/install/uninstall/detail-activate/reload/registry-status`、`api-plugin-setting-*`。
- 拡張点 (ミックスイン): APICall, Allocate, App, CurrencyExchange, DataExport, MachineDriver, Navigation, Notification, Report, Schedule, Settings, Transition, Validation, Barcode, Label, Locate, Action, UI

(`plugin/base/integration/`, `plugin/base/`)。

7. メッセージ一覧 (主要)

ID (発生源)	文言	発生条件
part-ipn-dup (`part/models.py:1043`)	Duplicate IPN not allowed in part settings	IPN 重複、かつ重複許可設定が無効
part-name-dup (`part/models.py:1086`)	Part with this Name, IPN and Revision already exists.	(name, IPN, revision) 重複
part-rev-dup (`part/models.py:1053`)	Duplicate part revision already exists.	同一 revision_of に同一 revision
part-rev-self (`part/models.py:780`)	Part cannot be a revision of itself	revision_of に自身を指定
part-rev-code (`part/models.py:786`)	Revision code must be specified for a part marked as a revision	改訂に revision 未設定
part-ipn-regex (`part/models.py:775`)	IPN must match regex pattern {pattern}	IPN がパターン不一致
part-category (`part/models.py:1087`)	Parts cannot be assigned to structural part categories!	構造カテゴリへの割当
part-locked (`part/models.py:593`)	Cannot delete this part as it is locked	locked 部品の削除
part-active (`part/models.py:596`)	Cannot delete this part as it is still active	active 部品の削除
part-in-bom (`part/models.py:600`)	Cannot delete this part as it is used in an assembly	BOM 使用中部品の削除
bom-recursive (`part/models.py:683`)	Part '{part}' cannot be used in BOM for '{parent}' (recursive)	BOM 循環参照
bom-locked (`part/models.py:3973`)	BOM item cannot be modified - assembly is locked	locked assembly の BOM 変更
bom-subpart (`part/models.py:4279`)	Sub part must be specified	sub_part 未指定
stock-location (`stock/models.py:1023`)	Stock items cannot be located into structural stock locations!	構造ロケーションへの配置
stock-serial-edit (`stock/models.py:1052`)	Editing of serial numbers is not allowed - this item has already been assigned a serial number	シリアル変更 (設定で禁止時)
stock-qty-int (`stock/models.py:1064`)	Quantity must be integer value for trackable parts	trackable 部品に小数数量
stock-virtual (`stock/models.py:1070`)	Stock item cannot be created for virtual parts	virtual 部品の在庫作成
stock-qty-zero (`stock/models.py:1079`)	Quantity must be greater than zero	数量が負 / 0 以下
stock-serial-qty (`stock/models.py:1095`)	Quantity must be 1 for item with a serial number	シリアル付き在庫の数量 > 1
stock-self (`stock/models.py:1122`)	Item cannot belong to itself	自己所属
stock-build-ref (`stock/models.py:1139`)	Build reference does not point to the same part object	出力 Part 不一致
stock-serial-dup (`stock/models.py:731`)	The following serial numbers already exist or are invalid : …	シリアル重複・不正
stock-missing (`stock/models.py:1494`)	Stock item no longer exists	調整対象が存在しない
stock-serial-del (`stock/models.py:582`)	Serialized stock items cannot be deleted	シリアル在庫の削除
build-external (`build/models.py:220`)	Build orders can only be externally fulfilled for purchaseable parts	非 purchaseable の外部委託
build-responsible (`build/models.py:230`)	Responsible user or group must be specified	責任者必須設定時に未指定

ID (発生源)	文言	発生条件
build-part (`build/models.py:235`)	Build order part cannot be changed	作成後の対象部品変更
build-date (`build/models.py:240`)	Target date must be after start date	target_date ≦ start_date

文言は `gettext_lazy` により翻訳される (`src/backend/InvenTree/locale/`)。上表は英語原文 (msgid)。

8. 移行・実装時に保持すべき要素

区分	保持すべき内容	根拠
状態コード値	BuildStatus / 各 OrderStatus / StockStatus / StockHistoryCode の**数値**と表示名 (API・レポート・外部連携が数値に依存)	`*/status_codes.py`
在庫可用性	「利用可能」= OK/Attention/Damaged/Returned の集合定義	`stock/status_codes.py:35-40`
API バージョン	`INVENTREE_API_VERSION` の単調増加と `INVENTREE_API_TEXT` の変更履歴記載	`InvenTree/api_version.py`
URL	`/api/<namespace>/` 構成と `api-*` ルート名、PUI の `/part/:id` 等の画面 URL (`get_absolute_url` が生成)	`InvenTree/urls.py`, `stock/models.py:1142`
採番	reference テンプレートによる自動採番、シリアル一意性スコープ設定、バッチコード生成エンドポイント	`build/models.py:265`, `stock/models.py:676`
入力順	製造: 発行 → 割当 → 出力生成 → 出力完了 → 完了 / 購買: 発行 → 入荷 → 完了	3.3 節の各 API
権限モデル	11 ルールセット × view/add/change/delete	`users/ruleset.py:41-60`
履歴	在庫操作は必ず `StockItemTracking` に追記 (削除・更新しない)	`stock/models.py:3716`
エラー文言	7 章の msgid (翻訳カタログのキー)	`locale/`
プラグイン契約	ミックスイン名・メソッドシグネチャ (外部プラグインが依存)	`plugin/base/integration/`

9. 対象外・既知の問題・前提・自己判断事項

9.1 対象外 (言及のみ)

- ドキュメントサイト (`docs/docs/`, `mkdocs.yml`)、CI/CD 定義 (`.github/workflows/`)、配布パッケージ (`contrib/`)、翻訳リソース (`locale/`)。
- テストコード (`*/test_*.py`, `src/frontend/tests/`) は仕様の根拠としてのみ参照し、機能として記述しない。
- レガシー UI テンプレート (`templates/`) は PUI 移行済みのため詳細化しない。

9.2 既知の問題 / 実装メモ

- `StockItem.clean()` に「clean 中に self.part が存在しないことがある (主に単体テスト時)」旨の未解決コメントがあり、例外を握り潰している (`stock/models.py:1071-1076`)。
- シリアル一意性は DB 制約ではなくアプリ側ロックで担保している (Part.tree_id を参照するため UniqueConstraint にできない、`stock/models.py:676-692`)。DB 直接投入では重複しうる。
- `Part.validate_unique()` は IPN / revision が `None` の場合 DB 制約と同様に重複を許す (Postgres の NULL 扱いに合わせた意図的挙動、`part/models.py:1057-1065`)。

9.3 前提・自己判断事項

1. 対象 ref は指定が無かったため master 最新コミット `c4c493d3` を採用した。
2. 実行環境への接続手段が無いため実機確認は未実施。全記述はソース根拠のみ (front matter `evidence`)。
3. 機能 ID (F-01〜F-31) は本書で新規に定義した。粒度は Django アプリ + 主要ユースケース単位とした。
4. 全モデル (バックエンド計 100 クラス超) の網羅は約 10 ページの制約に反するため、4 章は業務上の主要モデルに限定し、横断モデルは 1 行に集約した。

5. メッセージ一覧は代表的な検証エラー 28 件に限定した (全文言は `locale/` の翻訳カタログを参照)。
6. 状態遷移は各 API エンドポイント名から導出した。個別の遷移ガード条件の網羅は 5 章の業務ルールに委ねる。

10. 改訂履歴

日付	対象 ref	変更概要
2026-09-24	c4c493d3	初版作成 (master 最新コードベースに基づく全体仕様)